

3.5 基于 XML 数据集成的集成教务系统示例

近年来教育信息量的不断增多，使各级各类学校、家庭和教育管理部门对教育信息计算机和教育信息服务的要求越来越强烈。基于计算机在教育信息管理和信息服务显著的辅助作用，校园信息化在学校教育管理中日益普及。

原有三个独立的院系选课系统，地理上分散、结构上异质的。系统结构图如下图所示：

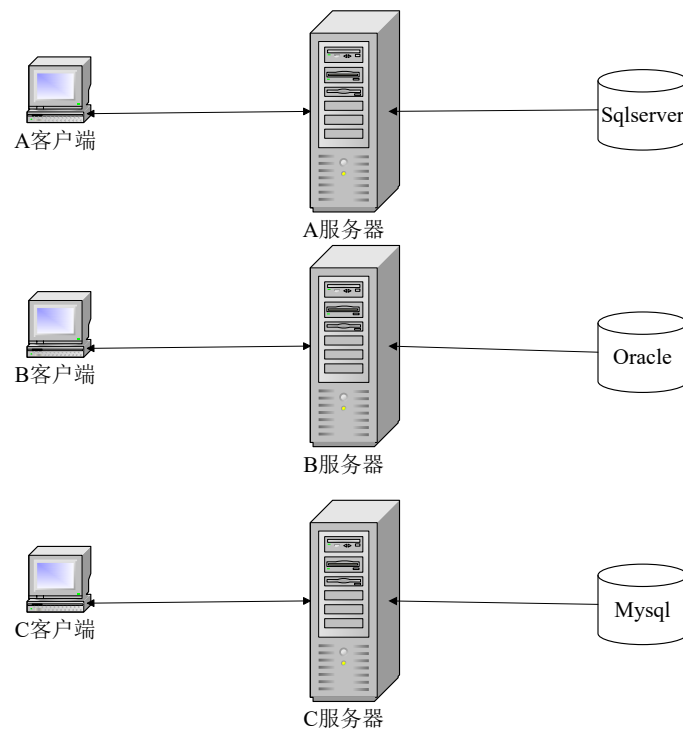


图 3-5：原教务系统系统结构图

原来的三个系统采用 C/S 结构或 B/S 结构，但选择不同的数据库以及不同的数据表结构和具体的程序逻辑结构各自独立实现了选课系统。

本项目基于原有的院系教务系统应用异构数据库集成技术，给出具有跨系选课功能的集成教务系统实例。集成教务系统项目通过将不同院系数据库的异构数据进行数据集成，分析和处理，使得用户在访问和处理不同院系的异构数据库中数据的同时，不必知道物理数据库的分布和结构，只需要类似本地查询和管理的操作。

使用数据集成技术将使系统具有以下特点：

1) 透明性

整个集成系统对客户端屏蔽了通信的细节，使得源系统在绝大部分不需要改动，保留各自系统的风格和使用方式。

2) 可扩展性

采用事实上通信格式的主臬 XML 作为通信数据的标准，使得集成之后的新系统更容易地去集成未来可能的其他教务系统，而且更大程度的复用已有的功能与代码。

3) 健壮性

XML 有许多成熟的产品、技术和应用，对已有的具有良好的性能的 XML 产品或开源成果的应用，系统将会得到更好的健壮性和良好的性能，并使得开发人员把注意力集中在业务逻辑的领域范畴内。

4) 安全性

在分布式的环境下，各个系统是相对独立的。它们的各自系统独立运作，保留各自的关键数据和信息，只暴露给外界需要暴露的数据。同时集成服务器也降低了各个系统服务器的访问负担。

3.5.1 原有教务系统分析

1. 原系统用例分析

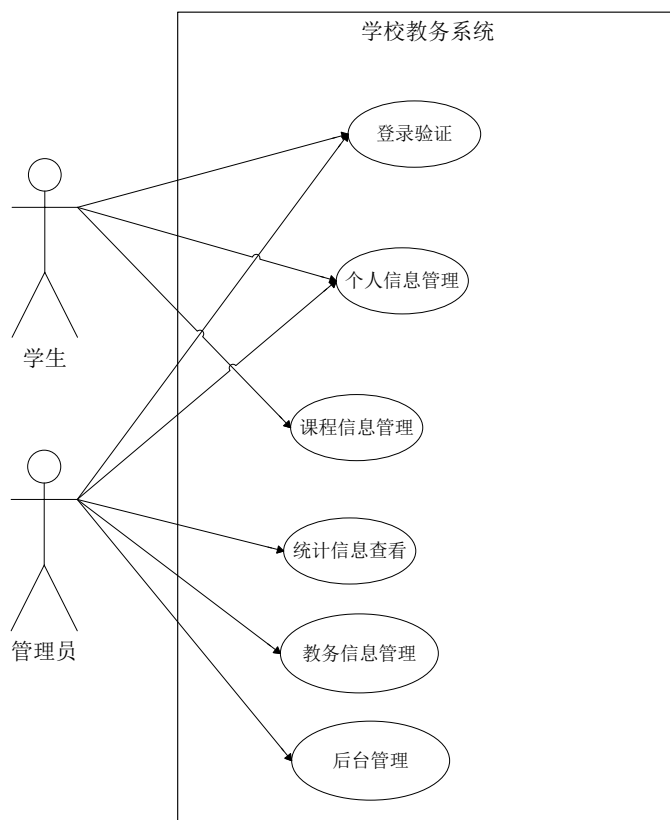


图 3-6：原教务系统用例图

对单个院系教务系统，用户可以包括学生，系统管理员，用户需求如下：

1) 对学生用户：

- ① 登陆验证：对不同年级的学生设置不同的选课权限
- ② 个人信息管理：个人信息查询更新
- ③ 个人课程管理：课程查看，选择，删除和更新

2) 对管理员用户：

- ① 登陆验证：对不同级别的管理员设置相应的管理权限
- ② 个人信息管理：个人信息查询更新
- ③ 统计信息查看：查看本院系学生、课程、课程被选情况等统计信息
- ④ 教务信息管理：对课程，开放课程，学生，选课等数据的设置操作
- ⑤ 后台管理：增删管理员，启动关闭服务器，查看在线情况等等

2. 原系统的异构性分析

1) 现有院系 A、院系 B 和院系 C 的教学管理系统基于不同的数据库系统。其中 A 的数据库系统使用 SqlServer, B 的数据库系统使用 Oracle, C 的数据库系统使用 MySQL 实现；其中每个数据库系统都包含了学生、课程以及选课的信息。

- 2) 院系 A、B、C 的学生互不覆盖，但课程信息有所重叠，选课规则有差别。
- 3) 院系 A、B 和 C 的学生、课程、选课信息的数据库结构有所差异（包括表结构、字段数量、字段名称、字段数据类型和数据意义等）。

3.5.2 集成教务系统分析与设计

1. 集成教务系统目标

集成教务系统的目标是满足课程共享的需求，使得各院系的学生能够跨院系选课。具体操作就是消除各系统数据异构性的影响，建立中间集成服务器，对不同院系数据库的异构数据进行数据集成，分析和处理，而同时对原有系统不需要对本地数据或代码做很大改变。

2. 集成系统用例分析

在原来的院系教务系统用例中，增加集成服务器的角色，集成服务器主要负责响应处理系统间的通信消息和命令，例如课程选择和更新，和接受与转发封装数据的 XML 文件，并进行 XML 文件的验证，转换。

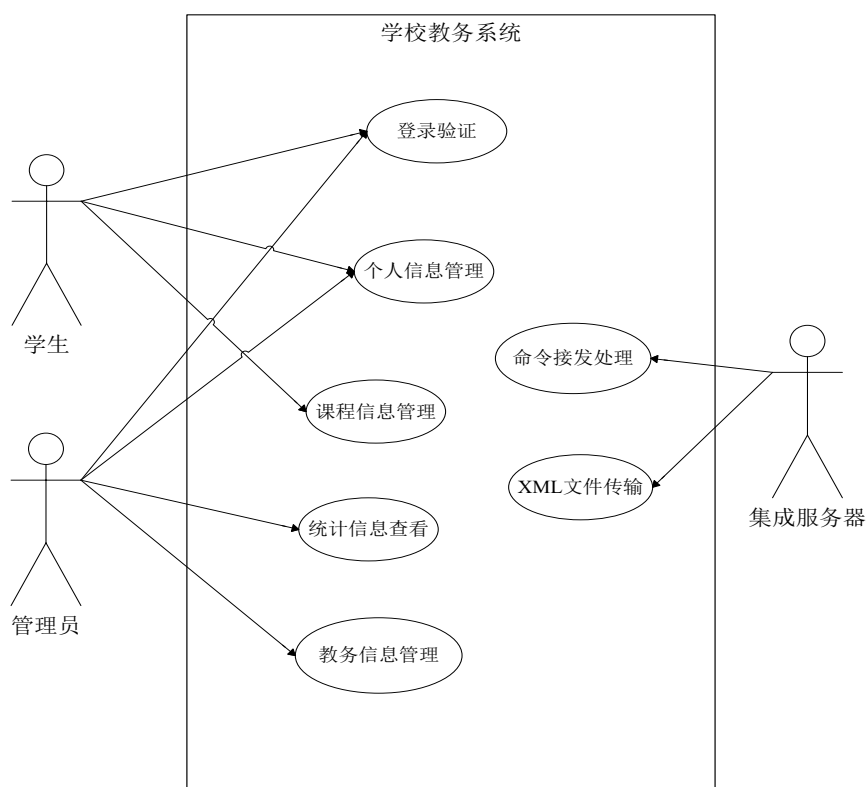


图 3-7：集成教务系统用例图

3. 集成系统架构设计

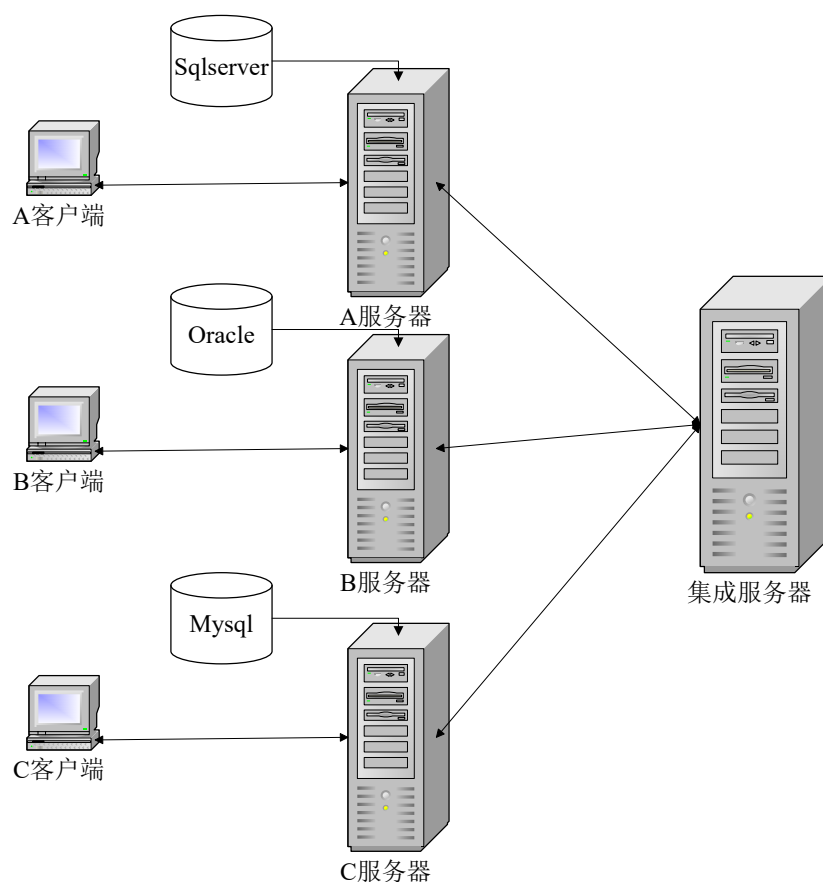


图 3-8: 集成教务系统架构设计

1) 集成端（集成服务器）

集成端包括如下几个部分：

UI，业务逻辑处理（如跨系选课，课程共享部分），XML 验证转换

工作流程如下：

① Server 接受各个教务系统 Server 发出的处理信号；

② XMLClient 处理与各个教务系统 XMLServer 之间通信，请求 XML 文件，获取 XML 文件；

③ 对数据进行逻辑处理，通过 XSLT 对各个源教务系统的 XML 格式文件进行转化，生成标准格式的 XML 文件；

④ 对标准格式的 XML 文件进行针对各个目标院系特定格式的转换，并将之回发到各个院系的服务器。

2) 教务系统（A、B、C 服务器）

① 原有的 LocalServer 端处理与本院系客户端的通信，如登录验证，课程信息传送等消息，使用自定义 socket 文本消息格式或使用 XML 标准格式表示查询或响应消息。

② 新增加的 XMLServer 端处理数据过程如下：通过读取本地数据库数据，将请求数据转化为 XML 文件，并将在与集成服务器的通信中使用。在处理 XML 文件时，用相应的 schema 进行验证。

3) 客户端（A、B、C 客户端）

各个院系的客户端应当提供具有登陆，课程查看，课程选择等其他个人管理功能。且只与相应院系的服务器端通信。

3.5.3 数据设计

1. 术语定义

XSL: 是一种标记语言, 表示如何将 xml 文档的内容装换成另一种形式的文档。

XML Schema: XML Schema 描述了 XML 文档的结构。可以用一个指定的 XML Schema 来验证某个 XML 文档, 以检查该 XML 文档是否符合其要求。

DOM4J: 是 dom4j.org 出品的一个开源 XML 解析包, 一个 Java 的 XML API, 类似于 Jdom, 用来读写 XML 文件。

Xerces: 一个开放源代码的 XML 语法分析器。

2. 数据库设计

1) 原有数据库设计

由学生, 账户, 课程, 选课四个关系表组成。学生与账户为一对一映射, 学生与课程为多对多映射, 其元祖将存储到选课表内。账户表中包括管理员账户。一个简化的数据 E-R 图如下:

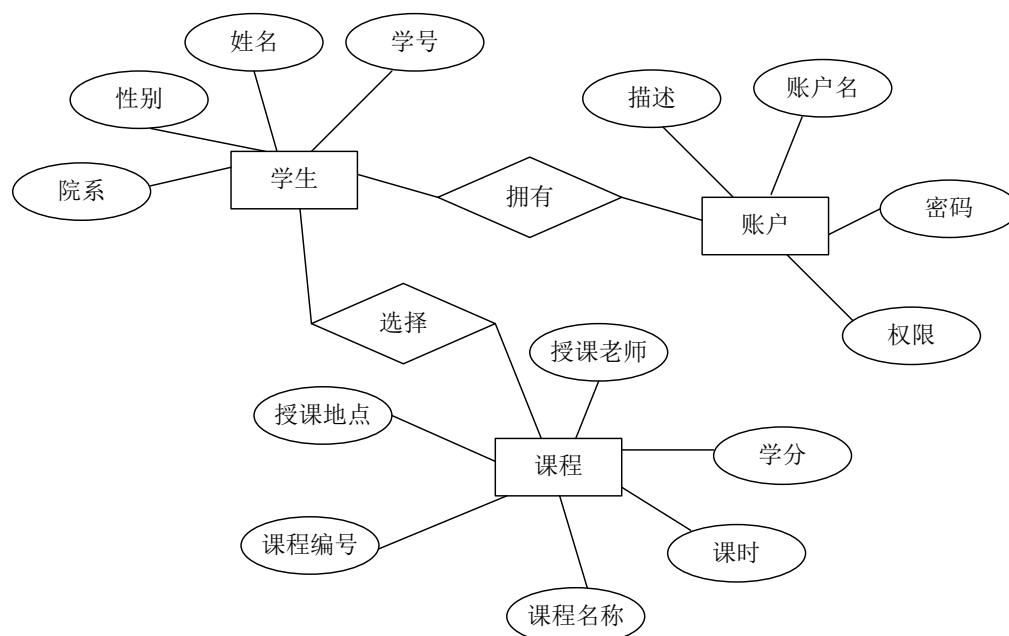


图 3-9: 原教务系统 E-R 图

2) 现数据库设计

考虑到院系课程有的可以设为共享, 有的课程不计划共享, 还有其他院系的共享课程, 所以在课程表中增加共享标记列, 标明课程类型。

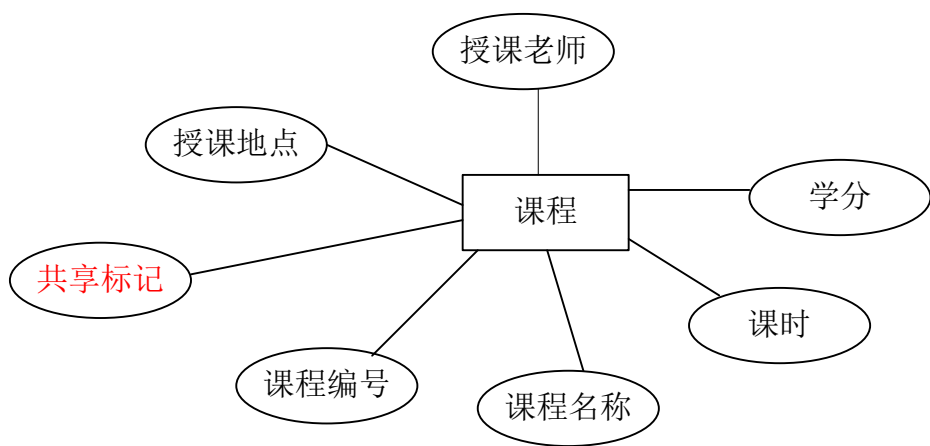


图 3-10：集成教务系统课程属性图

实例中数据库主要表结构设计：

1) 院系 A：

表 3-2：院系 A 账户表结构

字段	类型	约束
账户名	varchar(10)	PK
密码	varchar(6)	
权限	Char(4)	

表 3-3：院系 A 学生表结构

字段	类型	约束
学号	varchar(12)	primary key
姓名	varchar(10)	
性别	varchar(2)	
院系	varchar(10)	
关联账户	varchar(10)	foreign key

表 3-4：院系 A 课程表结构

字段	类型	约束
课程编号	varchar(8)	primary key
课程名称	varchar(10)	
学分	varchar(2)	
授课老师	varchar(10)	
授课地点	varchar(20)	
共享	char(1)	

表 1.3

表 3-5：院系 A 选课表结构

字段	类型	约束
课程编号	varchar(8)	unique(课程编号， 学生编号)
学生编号	varchar(12)	
成绩	varchar(3)	

2) 院系 B:

表 3-6: 院系 B 账户表结构

字段	类型	约束
账户名	varchar2(12)	primary key
密码	varchar2(12)	
级别	number(2)	
客体	varchar2(9)	foreign key

表 3-7: 院系 B 学生表结构

字段	类型	约束
学号	varchar2(9)	primary key
姓名	varchar2(10)	
性别	varchar2(2)	
专业	varchar2(16)	
密码	varchar2(6)	

表 3-8: 院系 B 课程表结构

字段	类型	约束
编号	varchar2(5)	primary key
名称	varchar2(16)	
课时	varchar2(2)	
学分	varchar2(1)	
老师	varchar2(10)	
地点	varchar2(20)	
共享	char(1)	

表 3-9: 院系 B 选课表结构

字段	类型	约束
课程编号	varchar2(5)	primary key
学号	varchar2(9)	
得分	varchar2(3)	

3) 院系 C:

表 3-10: 院系 C 账户表结构

字段	类型	约束
acc	varchar(12)	primary key
passwd	varchar(12)	
CreateDate	Timestamp	

表 3-11：院系 C 学生表结构

字段	类型	约束
Sno	varchar(9)	primary key
Snm	varchar(10)	
Sex	varchar(1)	
Sde	varchar(6)	
Pwd	char(6)	

表 3-12：院系 C 课程表结构

字段	类型	约束
Cno	char(4)	primary key
Cnm	varchar(10)	
Ctm	integer	
Cpt	integer	
Tec	varchar(20)	
Pla	varchar(18)	
Share	char(1)	

表 3-11：院系 C 选课表结构

字段	类型	约束
Cno	char(4)	foreign key
Sno	char(9)	foreign key
Grd	integer	

3. 数据集成应用相关XML定义

1) Schema 定义

编写每次转化需要的 XSD 文件。所需文件、部署位置及作用如下表格：

表 3-12：系统 Schema 文件列表

文件名称	部署位置	作用
formatClass.xsd	集成服务器	统一课程格式验证
formatStudent.xsd	集成服务器	统一学生格式验证
formatClassChoice.xsd	集成服务器	统一选课格式验证
studentA.xsd	A 服务器	A 学生格式验证
studentB.xsd	B 服务器	B 学生格式验证
studentC.xsd	C 服务器	C 学生格式验证
classA.xsd	A 服务器	A 课程格式验证
classB.xsd	B 服务器	B 课程格式验证
classC.xsd	C 服务器	C 课程格式验证
choiceA.xsd	A 服务器	A 选课格式验证
choiceB.xsd	B 服务器	B 选课格式验证
choiceC.xsd	C 服务器	C 选课格式验证

如图所示是集成服务器所使用的统一数据 XML 的模式的具体定义。

课程元素定义：

表 3-13：集成服务器端课程元素定义列表

元素/属性	定义	类型	限定	父元素
classes	所有课程	复合元素	必选	root
class	单个课程	复合元素	minOccurs=1	classes
id	课程编号	string	Length=9	class
name	课程名称	string	maxLength=40	class
time	课时	unsignedByte	无	class
score	学分	unsignedByte	maxInclusive=5	class
teacher	授课教师	string	maxLength=20	class
location	授课地点	string	maxLength=40	class
...	...	...	...	...

表 3-14：集成服务器端学生元素定义列表

元素/属性	定义	类型	限定	父元素
students	所有学生	复合元素	必选	root
student	单个学生	复合元素	minOccurs=1	students
id	学号	string	maxLength=9	student
name	学生姓名	string	maxLength=40	student
major	专业	string	maxLength=40	student
...	...	...	...	...

表 3-15：集成服务器端选课元素定义列表

元素/属性	定义	类型	限定	父元素
choices	所有选课	复合元素	必选	root
choice	单个选课	复合元素	minOccurs=1	choices
sid	学生学号	string	maxLength=12	choice
cid	课程编号	string	Length=9	choice
score	成绩	unsignedByte	maxInclusive=100	choice
...	...	...	...	...

下面为 formatStudent.xsd 的示例。

```

<?xml version="1.0" encoding="gb2312"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="xml" encoding="gb2312"/>
  <xsl:template match="Students">
    <xsl:apply-templates/>
    <Students>
      <xsl:for-each select="student">
        <student>
          <xsl:choose>
            <xsl:when test="A 条件">
              <id>
                <xsl:value-of select="****"/>
              </id>
            </xsl:when>
            <xsl:when test=" B 条件">
              <id>
                <xsl:value-of select="****"/>
              </id>
            </xsl:when>
            <xsl:when test="C 条件">
              <id>
                <xsl:value-of select="****"/>
              </id>
            </xsl:when>
          </xsl:choose>
          <name>
            <xsl:value-of select="姓名"/>
            <xsl:value-of select="Snm"/>
          </name>
          <sex>
            <xsl:value-of select="性别"/>
            <xsl:value-of select="Sex"/>
          </sex>
          <xsl:choose>
            <major>
              <xsl:value-of select="专业"/>
            </major>
          </xsl:choose>
        </student>
      </xsl:for-each>
    </Students>
  </xsl:template>
</xsl:stylesheet>

```

图 3-11: formatStudent.xsd 示例

2) XSL 定义

所有文件都部署在集成服务器上，所需文件和作用如下表格所示：

表 3-16：XSL 文件部署

文件名称	作用
formatClass.xsl	各系统课程格式转换到统一课程格式
formatStudent.xsl	各系统学生格式转换到统一学生格式
formatClassChoice.xsl	各系统选课格式转换到统一选课格式
studentToA.xsl	统一学生格式转换到 A 学生格式
studentToB.xsl	统一学生格式转换到 B 学生格式
studentToC.xsl	统一学生格式转换到 C 学生格式
classToA.xsl	统一课程格式转换到 A 课程格式
classToB.xsl	统一课程格式转换到 B 课程格式
classToC.xsl	统一课程格式转换到 C 课程格式
choiceToA.xsl	统一选课格式转换到 A 选课格式
choiceToB.xsl	统一选课格式转换到 B 选课格式
choiceToC.xsl	统一选课格式转换到 C 选课格式

都部署在集成服务器端，用来转换各个院系教务系统发过来的学生，课程，选课 XML 数据到统一格式的 XML，再将统一格式的 XML 转换成目标院系教务系统的期望 XML 格式。

如下图课程的统一格式 XML 到各院系教务系统课程 XML 格式的转换映射：

表 3-17：课程元素映射表

统一格式 XML 元素	A 系统 XML 元素	B 系统 XML 元素	C 系统 XML 元素
id	课程编号	编号	Cno
name	课程名称	名称	Cnm
score	学分	学分	Cpt
teacher	授课老师	老师	Tec
location	授课地点	地点	Pla

学生的统一格式 XML 到各院系教务系统学生 XML 格式的转换映射：

表 3-18：学生元素映射表

统一格式 XML 元素	A 系统 XML 元素	B 系统 XML 元素	C 系统 XML 元素
id	学号	学号	Sno
name	姓名	名臣	Snm
sex	性别	性别	Sex
major	院系	专业	Sde

选课的统统一格式 XML 到各院系教务系统选课 XML 格式的转换映射：

表 3-19：选课元素映射表

统一格式 XML 元素	A 系统 XML 元素	B 系统 XML 元素	C 系统 XML 元素
sid	学号	学生编号	Sno
cid	课程编号	课程编号	Cno
score	成绩	得分	Grd

下面为统一格式到 classToA.xsl 的示例:

```
<? xml    version= "1.0"    encoding= "gb2312"?>
<xsl:stylesheet    version= "1.0"    xmlns:xsl= "http://www.w3.org/1999/XSL/Transform">
  <xsl:output    method= "xml"    encoding= "gb2312"/>
  <xsl:template match="Classes">
    <xsl:apply-templates/>
    <Classes>
      <xsl:for-each select="class">
        <class>
          <课程编号>
            <xsl:value-of select="id"/>
          </课程编号>
          <课程名称>
            <xsl:value-of select="name"/>
          </课程名称>
          <学分>
            <xsl:value-of select="score"/>
          </学分>
          <授课老师>
            <xsl:value-of select="teacher"/>
          </授课老师>
          <授课地点>
            <xsl:value-of select="location"/>
          </授课地点>
        </class>
      </xsl:for-each>
    </Classes>
  </xsl:template>
</xsl:stylesheet>
```

图 3-12: classToA.xsl 示例

3.5.4 重点流程解析

◆ 课程共享

A 院系申请课程共享，通信过程如下图所示：

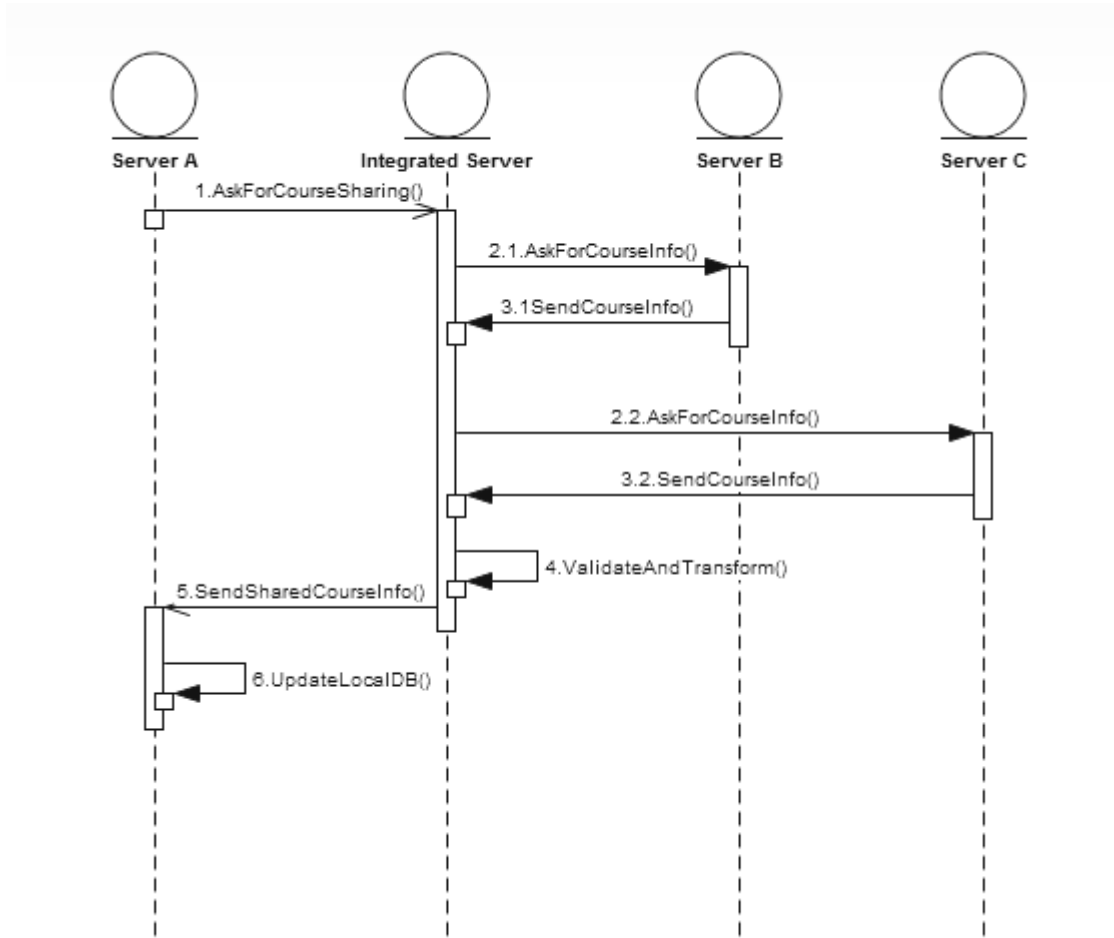


图 3-13：课程共享流程

课程共享步骤如下：

1. 在 A 院系教务系统服务器端，用户点击【执行课程共享】按钮，向集成服务器发送课程共享请求。
2. 集成服务器解析请求，向其他院系教务系统服务器发送对本院系课程信息的请求。
3. B,C 教务系统服务器端通过读取各自的数据库中课程表的数据，并生成 XML 文件，将之返回给集成服务器端。
4. 集成服务器端将各个院系发送过来的 XML 数据利用针对的 XSL 文件转换成统一的格式，在此基础上，将需要发往 A 教务系统的数据重新转换成对应 A 院系特定格式的 XML 文件。
5. 将综合的课程信息并返回给请求的 A 教务系统的服务器。
6. A 院系解析收到的共享课程 XML 数据，并存储到各自院系的数据库中。

◆ 选课流程：

如果 A 院系的学生选择 A 院系的课程，则向服务器发送消息，服务接收选课消息并在数据库中作出插入删除或更新。

如果 A 院系的学生选择 B 院系的课程，则向服务器发送消息，服务器检测到选择的不是本院系课程，则将选课信息发送至集成服务器。集成服务器解析消息，并将

选课消息转发到相应的 B 教务系统服务器。B 教务系统服务器接收消息后，分析处理并在相应的数据库中记录选课信息。

例：A 院系学生 S 选择 B 系 X 课程：

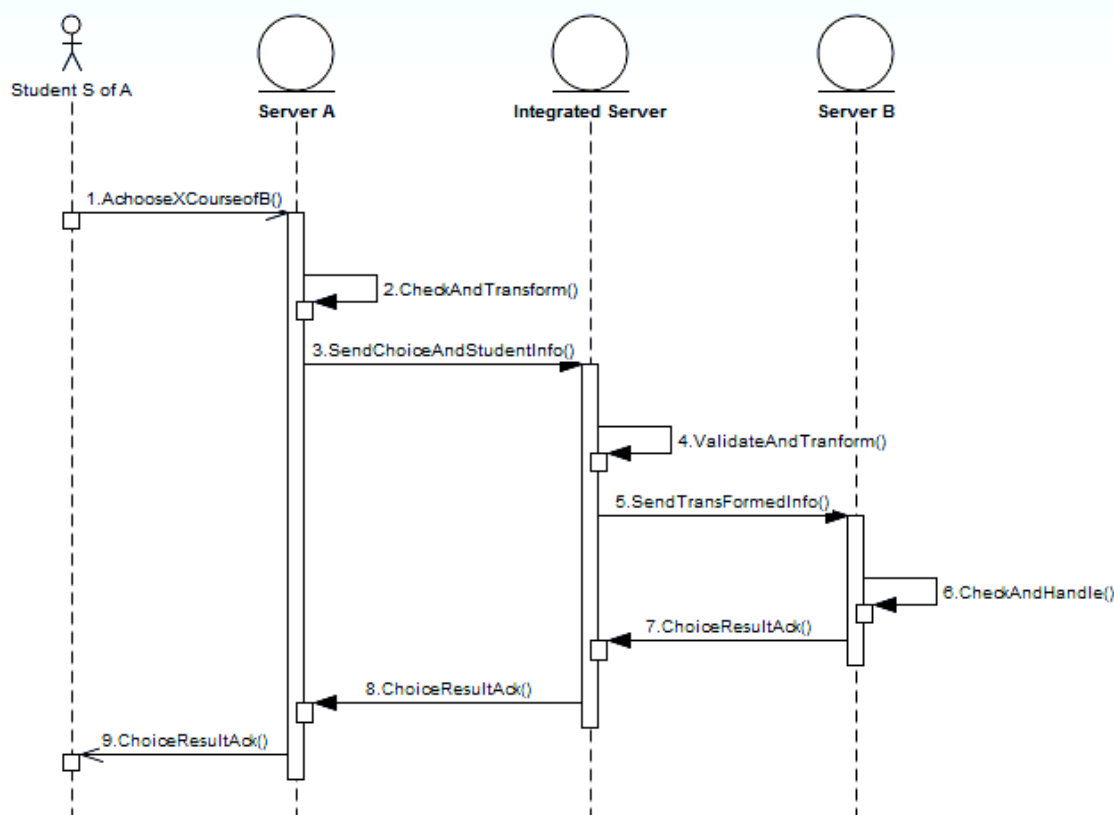


图 3-14：跨系选课流程

选课步骤：

1. A 院系 S 学生发送对 B 课程的选课请求。
2. 服务器检测出是跨系选课，封装消息成统一查询格式。
3. 向集成服务器发送对 B 课程 X 的选课请求和 S 学生信息。
4. 对接收过来的课程和学生 XML 数据进行验证和转换。
5. 向 B 院系教务系统服务器发送经过转换的 X 选课和学生信息。
6. 检查 S 选择 X 课程的有效性，决定是否将 S 学生信息更新到自己的数据库。
7. 向集成服务器发送 S 选课是否成功的确认信息。
8. 集成服务器向 A 教务系统服务器发送 S 选课结果的确认信息。
9. 服务器向 S 学生选课是否成功。

3.5.5 数据集成关键代码实现

1. XML生成

该模块主要负责从院系数据库中读取数据，并生成本地格式的 XML 文件，部分代码示例如下：

使用 DOM4J 生成 xml 文件

```

import org.dom4j.Document;
import org.dom4j.DocumentHelper;
import org.dom4j.Element;

// 任务:从 resultset 结果集中提取元数据信息
ResultSetMetaData rsmd = rs.getMetaData();
// 表的字段数目
int count = rsmd.getColumnCount();
// 存入列名数组
String[] columnName = new String[count];
for (int i = 1; i <= count; i++) {
    columnName[i - 1] = rsmd.getColumnName(i);
}
//建立 document 对象
doc = DocumentHelper.createDocument();
Element root = doc.addElement("Students");
while (rs.next()) {
    Element emp = root.addElement("student");
    for (int i = 1; i <= count; i++) {
        Element column = emp.addElement(columnName[i - 1]);
        if (rs.getObject(i) != null) column.setText(rs.getObject(i) + "");
        else column.setText("");
    }
}
}

```

图 3-15：使用 DOM4J 生成 xml 的代码

将 document 中的内容写入文件中

```

Writer w = new FileWriter("D:\\\" + outputName + ".xml");
OutputFormat opf = OutputFormat.createPrettyPrint();
opf.setEncoding("GB2312");
XMLWriter xw = new XMLWriter(w, opf);
xw.write(doc);
xw.close();
w.close();

```

图 3-16：使用 DOM4J 生成 xml 的代码

2. XML 验证

利用本地的 XSD 文件对发送过来的 XML 文件进行验证，sax 部分使用 xercesImpl.jar 包。部分示例代码如下：

```

SAXReader saxReader = new SAXReader();
saxReader.setValidation(true);
saxReader.setFeature("http://xml.org/sax/features/validation", true);
saxReader.setFeature("http://apache.org/xml/features/validation/schema", true);

saxReader.setProperty("http://apache.org/xml/properties/schema/external-noNamespaceSchemaLocation", "*.xsd");

XMLErrorHandler errorHandler = new XMLErrorHandler();
saxReader.setErrorHandler(errorHandler);

Document document = saxReader.read(new FileReader(new File("D:\\"+ fileName + ".xml")));

XMLWriter writer = new XMLWriter(OutputStream.createPrettyPrint());
if (errorHandler.getErrors().hasContent()) {
    writer.write(errorHandler.getErrors());
} else {
    System.out.println("validate success.");
}

```

图 3-17：基于 sax 验证 XML 文件的代码

3. XML转换

利用本地的 XSL 文件对发送过来的 XML 文件转换成目标格式，部分示例代码如下：

```

SAXReader saxReader = new SAXReader();
Document document = saxReader.read(new FileReader(new File("*.xml")));
TransformerFactory factory = TransformerFactory.newInstance();
Transformer transformer = factory.newTransformer(new StreamSource(styleSheet));
DocumentSource source = new DocumentSource(document);
DocumentResult result = new DocumentResult();
=====XML 转换开始=====
transformer.transform(source, result);
Document transformedDoc = result.getDocument();
Writer w = new FileWriter("**.xml");
OutputStream opf = OutputStream.createPrettyPrint();
opf.setEncoding("GB2312");
XMLWriter xw = new XMLWriter(w, opf);
xw.write(transformedDoc);
// 关闭文件流
xw.close();
w.close();

```

图 3-18：利用 XSL 转换 XML 的代码

4. XML解析

XML 的解析有多种技术，本示例中采用了面向文档的对象式解析技术 DOM 和面向文

档的流式解析技术 SAX。DOM（例为 dom4j 解析代码）：

```
//=====开始分析 XML=====
Connection con = new DataBaseConnection().getConnection();
PreparedStatement ps = null;
int rs = 0;
Document doc = null;
SAXReader saxReader = new SAXReader();
Document document = saxReader.read(new FileReader(new File("*.xml")));
Element root = document.getRootElement();
for (Iterator i = root.elementIterator(); i.hasNext();) {
    Element foo = (Element) i.next();
    Vector<String> employee = new Vector<String>();
    for (Iterator j = foo.elementIterator(); j.hasNext();) {
        Element tmp = (Element) j.next();
        employee.add(tmp.getStringValue());
    }
    Vector<String> allName = new Vector<String>();
    String sql = "insert into ** values('"+ employee.elementAt(0);
    for (int k = 1; k < employee.size(); k++)
        sql += "','" + employee.elementAt(k);
    sql += "'";
    ps = con.prepareStatement(sql);
    rs = ps.executeUpdate();
}
//=====结束分析 XML=====
```

图 3-19： 解析 XML 的代码